连接树(左深连接树、右深连接树、浓密树)

Improving Logical Query Plans
Optimization of Join Orders

连接顺序的优化

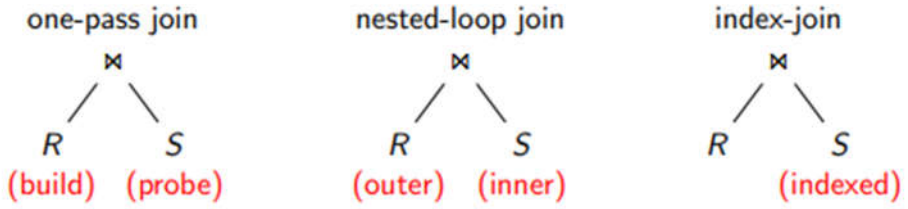
优化连接操作的执行顺序对于提高查询执行效率至关重要

- 连接操作的执行代价高
- 在数据库上执行几十个关系的连接很常见，如联机在线分析(Online Analytical Processing, OLAP)任务
- 不同连接顺序的执行代价差异巨大

连接关系的角色

尽管连接操作满足交换律，但在查询计划中连接操作的输入关系具有不同的作用

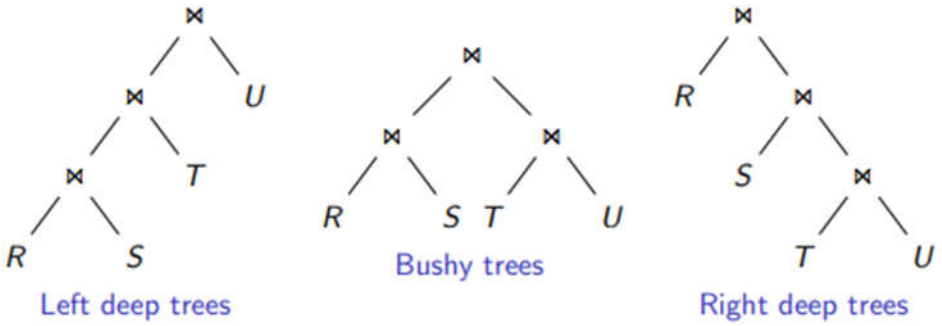
- 在$R \bowtie S$中，R是左关系(left relation)，S是右关系(right relation)
- 如果使用一趟连接(one-pass join)，则左关系是构建关系(build relation)，右关系是探测关系(probe relation)
- 如果使用嵌套循环连接(nested-loop join)，则左关系是外关系(outer relation)，右关系是内关系(inner relation)
- 如果使用索引连接(index-based join)，则右关系是有索引的关系(indexed relation)



连接树(Join Trees)

一组关系上的连接操作的执行顺序可以用连接树(join tree)来表示

- 左深连接树(left-deep join tree): 只有一个关系是左关系(left relation)，其他关系都是右关系(right relation)
- 右深连接树(right-deep join tree): 只有一个关系是右关系，其他关系都是左关系(left relation)
- 浓密树(bushy tree): 除左深连接树和右深连接树以外的其他连接树



左深连接树(Left-Deep Join Trees)

System R的查询优化器只考虑左深连接树

优点1: 在给定关系上，全部左深连接树的数量比全部连接树少得多

- $R_1 \bowtie R_2 \bowtie \cdots \bowtie R_n$的全部左深连接树的数量为$n!$
- $R_1 \bowtie R_2 \bowtie \cdots \bowtie R_n$的全部连接树的数量为$n!C_n$，其中$C_n$表示含有$n$个叶子节点的树形的数量(Catalan数)

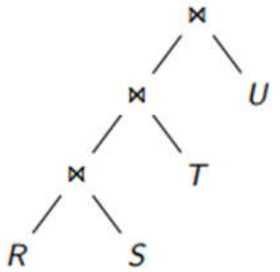
$$C_1 = 1,$$
$$C_n = \sum_{i=1}^{n-1} C_i C_{n-i} \qquad \text{if } n > 1$$

优点2: 基于左深连接树的查询计划通常比基于其他类型连接树的查询计划的执行效率更高

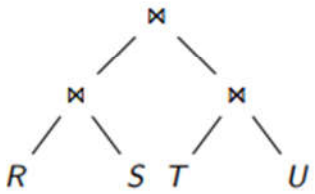
- 可能形成完全通畅的流水线(fully piplined plans)

为什么左深连接树查询计划更高效?

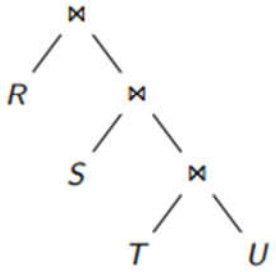
如果下列查询计划全部使用一趟连接(one-pass join)算法，那么在流水线查询执行模型(piplining model)下，左深连接树查询计划在任何时刻对内存的需求都比其他查询计划更低



At any time, only one of R, $R \bowtie S$, and $R \bowtie S \bowtie T$ will be held in the buffers



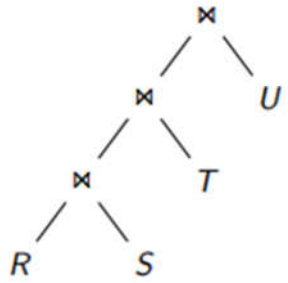
$R \bowtie S$ and T must be held in the buffers at the same time



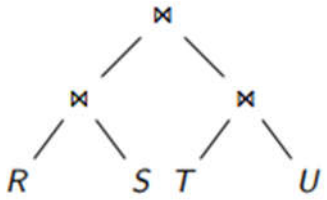
All of R, S, and T must be held in the buffers at the same time

为什么左深连接树查询计划更高效?

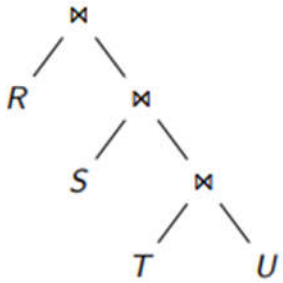
如果下列查询计划全部使用嵌套循环连接(nested-loop join)算法，那么在流水线查询执行模型(piplining model)下，左深连接树查询计划不需要多次构建中间关系



Both $R \bowtie S$ and $R \bowtie S \bowtie T$ are constructed only once



$R \bowtie S$ is constructed only once, but $T \bowtie U$ needs to be constructed more than once (if not materialized)



$S \bowtie T \bowtie U$ needs to be constructed more than once, so does $T \bowtie U$

优化连接顺序的动态规划方法

Example (连接顺序优化)

Choose the best join order for $R(a,b) \bowtie S(b,c) \bowtie T(c,d) \bowtie U(d,a)$

$R(a,b)$	$S(b,c)$	$T(c,d)$	$U(d,a)$
$T(R) = 1000$	$T(S) = 1000$	$T(T) = 1000$	$T(U) = 1000$
$V(R,a) = 100$			$V(U,a) = 50$
$V(R,b) = 200$	$V(S,b) = 100$		
	$V(S,c) = 500$	$V(T,c) = 20$	
		$V(T,d) = 50$	$V(U,d) = 1000$

Best orders of joins on 2 relations

	$R \bowtie S$	$R \bowtie T$	$R \bowtie U$	$S \bowtie T$	$S \bowtie U$	$T \bowtie U$
Est. size	5,000	1M	10,000	2,000	1M	1,000
Est. cost	0	0	0	0	0	0
Best plan	$R \bowtie S$	$R \bowtie T$	$R \bowtie U$	$S \bowtie T$	$S \bowtie U$	$T \bowtie U$

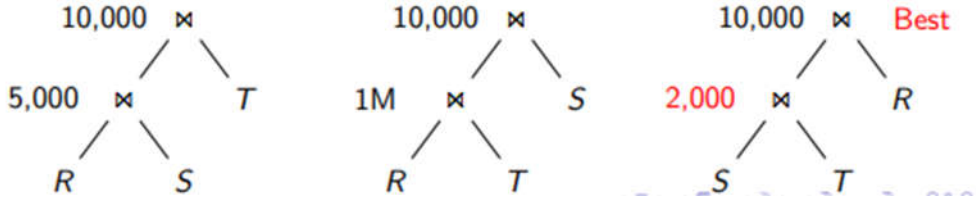
Best orders of joins on 2 relations

	$R \bowtie S$	$R \bowtie T$	$R \bowtie U$	$S \bowtie T$	$S \bowtie U$	$T \bowtie U$
Est. size	5,000	1M	10,000	2,000	1M	1,000
Est. cost	0	0	0	0	0	0
Best plan	$R \bowtie S$	$R \bowtie T$	$R \bowtie U$	$S \bowtie T$	$S \bowtie U$	$T \bowtie U$

Best orders of joins on 3 relations

	$R \bowtie S \bowtie T$	$R \bowtie S \bowtie U$	$R \bowtie T \bowtie U$	$S \bowtie T \bowtie U$
Est. size	10,000	50,000	10,000	2,000
Est. cost	2,000	5,000	1,000	1,000
Best plan	$(S \bowtie T) \bowtie R$	$(R \bowtie S) \bowtie U$	$(T \bowtie U) \bowtie R$	$(T \bowtie U) \bowtie S$

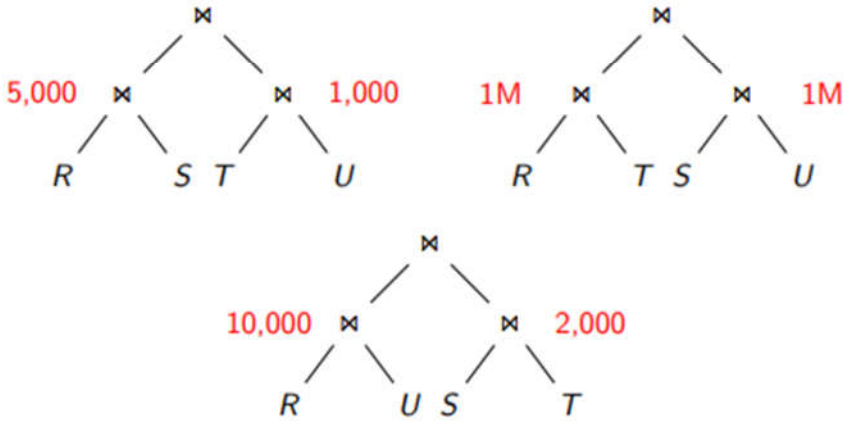
For example, the best plan for $R \bowtie S \bowtie T$ is obtained as follows



The best plan for $R \bowtie S \bowtie T \bowtie U$ is obtained as follows (phase 1)

Best orders of joins on 2 relations

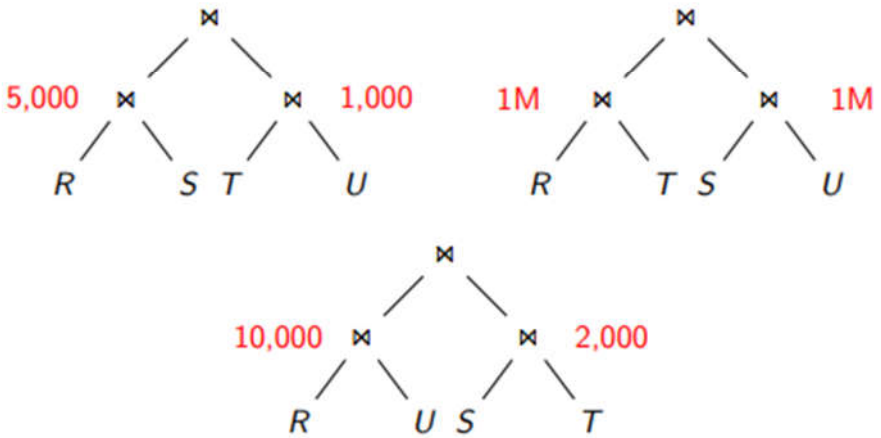
	$R \bowtie S$	$R \bowtie T$	$R \bowtie U$	$S \bowtie T$	$S \bowtie U$	$T \bowtie U$
Est. size	5,000	1M	10,000	2,000	1M	1,000
Est. cost	0	0	0	0	0	0
Best plan	$R \bowtie S$	$R \bowtie T$	$R \bowtie U$	$S \bowtie T$	$S \bowtie U$	$T \bowtie U$



The best plan for $R \bowtie S \bowtie T \bowtie U$ is obtained as follows (phase 1)

Best orders of joins on 2 relations

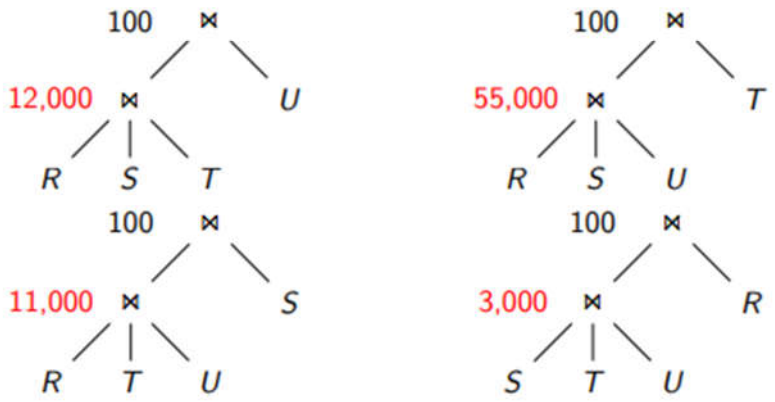
	$R \bowtie S$	$R \bowtie T$	$R \bowtie U$	$S \bowtie T$	$S \bowtie U$	$T \bowtie U$
Est. size	5,000	1M	10,000	2,000	1M	1,000
Est. cost	0	0	0	0	0	0
Best plan	$R \bowtie S$	$R \bowtie T$	$R \bowtie U$	$S \bowtie T$	$S \bowtie U$	$T \bowtie U$



The best plan for $R \bowtie S \bowtie T \bowtie U$ is obtained as follows (phase 2)

Best orders of joins on 3 relations

	$R \bowtie S \bowtie T$	$R \bowtie S \bowtie U$	$R \bowtie T \bowtie U$	$S \bowtie T \bowtie U$
Est. size	10,000	50,000	10,000	2,000
Est. cost	2,000	5,000	1,000	1,000
Best plan	$(S \bowtie T) \bowtie R$	$(R \bowtie S) \bowtie U$	$(T \bowtie U) \bowtie R$	$(T \bowtie U) \bowtie S$



查询执行模型：物化、流水线

查询计划的执行方法

如何执行由多个操作构成的查询计划(query plan)?

- 方法1: 物化执行(Materialization)
- 方法2: 流水线执行(Piplining)

物化执行(Materialization)

- 自底向上执行查询计划中的操作
- 每个中间操作的执行结果写入临时关系文件，作为后续操作的输入

Example (物化执行)

$\Pi_{ID,salary}$ - - - - - - - - - - - - → (输出关系文件)

|

$\sigma_{salary<75000}$; use index ← - - - - - - → (临时关系文件)

|

instructor ← - - - - - - - - - - - - (输入关系文件)

物化执行的缺点

缺点1: 物化(materialize)临时关系增加了查询执行的代价

- 执行完一个操作后，临时关系必须写入文件(除非临时关系非常小)
- 执行后续操作时，临时关系文件再被读入缓冲区

缺点2: 用户获得查询结果的时间延迟大

Example (物化执行)

$\Pi_{ID,salary}$ - - - - - - - - - - - → (Output Relation)

|

$\sigma_{salary<75000}$; use index ← - - - - → (Temprory Relation)

|

instructor ← - - - - - - - - - - - - (Input Relation)

流水线执行(Piplining)

将查询计划中若干操作组成流水线(pipline)，一个操作的结果直接传给流水线中下一个操作

- 避免产生一些临时关系，避免了读写这些临时关系文件的I/O开销
- 用户能够更快地得到查询结果

几乎所有DBMS都使用流水线执行查询计划

Example (流水线执行)

$\Pi_{ID,salary}$ - - - - - - - - - - - → (Output Relation)

| piplined

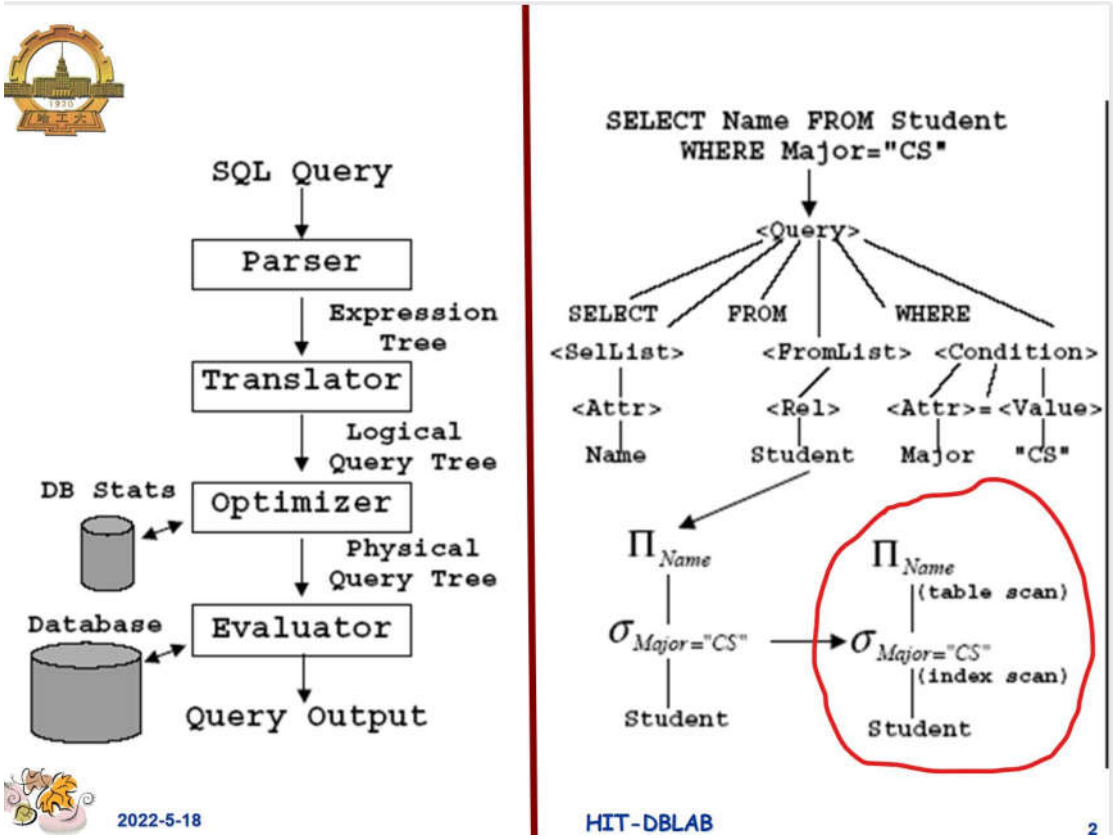
$\sigma_{salary<75000}$; use index (~~Temporary Relation~~)

|

instructor ← - - - - - - - - - - - - (Input Relation)

物理查询计划，

就是将 Scan 方法，循环嵌套查询，Hash-Join 等嵌入到逻辑查询树中，例如下图



关于即时更新技术

- 即时更新协议
  - T开始执行时，记录<T, start>被写到日志。
  - 在T运行期间，当T发出一个write(X)操作时，记录<T, X, v_1,v_2>首先被写入日志，然后直接在数据库上执行write(X)。
  - 当T部分提交时，记录<T, commit>被写到日志。

部分提交：<T,commit>写入日志，但数据库可能出现未更新完的情况，此时如果发生故障，这些部分提交的事务需要 redo

例子：

日志文件

<T0, start >
<T0, A, 1000, 950>

<T0, B, 2000, 2050>

<T0, commit >
<T1, start >
<T1, C, 700, 600>

<T1, commit >

此时 T1 commit，但是存在一种可能是 T1 还没有将 C 更新，所以此时需要对 T1

进行 Redo

第六次作业答案

1.设教学管理数据库有如下 3 个关系模式：

S(S#, SNAME, AGE, SEX)

C(C#, CNAME, TEACHER)

SC(S#, C#, GRADE)

其中 S 为学生信息表、SC 为选课表、C 为课程信息表；S#、C#分别为 S、C 表的主码，(S#, C#)是 SC 表的主码，也分别是参照 S、C 表的外码

用户有一查询语句：

Select SNAME

From S, SC, C

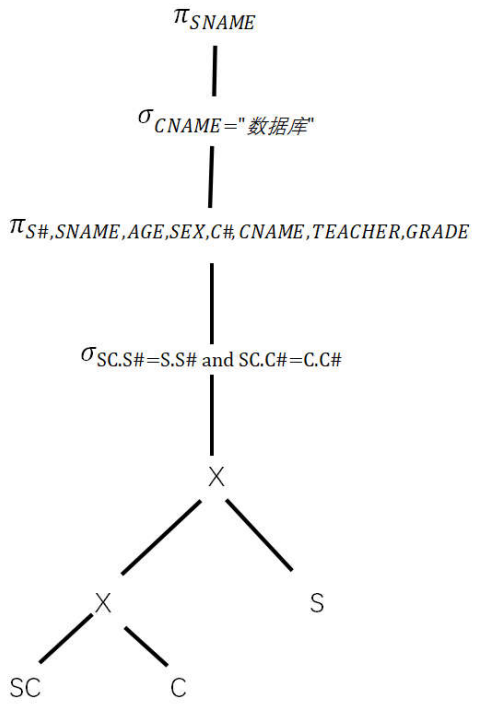
Where SC.S#=S.S# and SC.C#=C.C# and CNAME= “数据库”

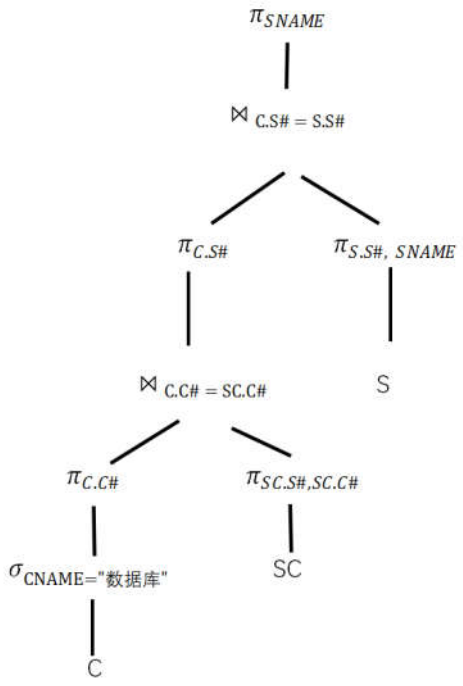
检索选学“数据库”课程的学生的姓名。

(1)写出以上 SQL 语句所对应的关系代数表达式。

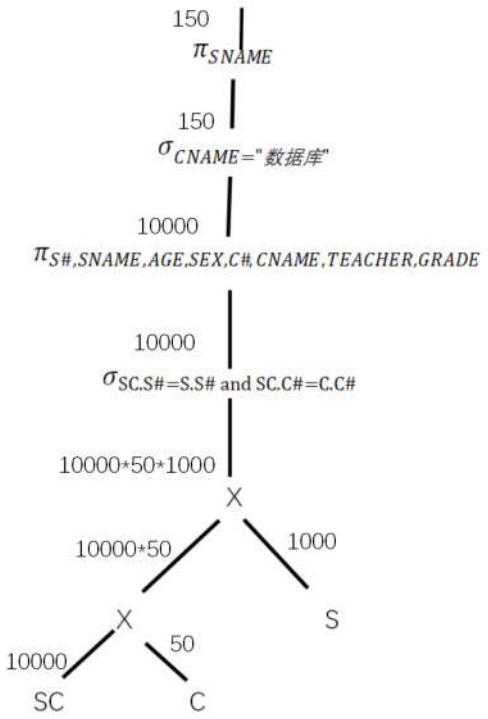
$$\Pi_{SNAME}(\sigma_{CNAME=\text{"数据库"}\wedge SC.S\#=S.S\#\wedge SC.C\#=C.C\#}(SC \times C \times S))$$

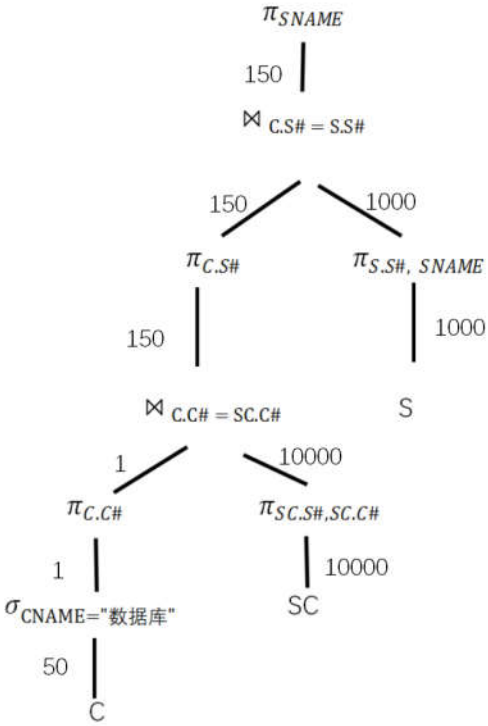
(2)画出上述关系代数表达式所对应的查询计划树。使用启发式查询优化算法，对以上查询计划树进行优化，并画出优化后的查询计划树。





(3)设 SC 表有 10000 条元组，C 表有 50 条元组，S 表中有 1000 条元组，SC 中满足选修数据库课程的元组数为 150，计算优化前与优化后的查询计划中每一步所产生的中间结果大小





2、某个数据库的一个关系模式如下，其中主码用下划线标出
Branch(<u>branch_name</u>, branch_city, assets)

考虑下面的 SQL 语句：
select T.branch_name
from branch T,branch S
where T.assert>S.assert **and** S.branch_city=’’BeiJing’’

(1) 写出以上 SQL 语句所对应的关系代数表达式.

$$\Pi_{T.branch_name}(\sigma_{T.assert>S.assert\ \wedge\ S.branch_city=''BeiJing''}(\rho_T(\text{branch}) \times \rho_S(\text{branch}))$$

(2) 写出一个与（1）中的关系代数表达式等价的，但更高效的关系代数表达式。

$$\Pi_{T.branch_name}\left((\Pi_{T.branch_name,T.assets}(\rho_T(\text{branch}))\right) \bowtie_{T.assert>S.assert} (\Pi_{S.assets}\left(\sigma_{S.branch_{city=''BeiJing''}}(\rho_S(\text{branch}))\right))$$

3、考虑下面两个事务：

T_1:read(A)
read(B)
if A=0 then B:=B+1
write(B);

T_2:read(B)
read(A)
if B=0 then A:=A+1
write(A).

给事务 T1、T2 增加加锁和解锁指令，使它们遵从两段锁协议。这两个事务会引起死锁吗？如果会，请举出例子。

加锁和解锁指令:

T_1 :
lock-S(A)
read(A)
lock-X(B)
read(B)
if A = 0
then B := B + 1
write(B)
unlock(A)
unlock(B)

T_2:
lock-S(B)
read(B)
lock-X(A)
read(A)
if B = 0
then A := A + 1
write(A)
unlock(B)
unlock(A)

这些事务的执行可能会导致死锁。例如以下部分时间表：

T1	T2
lock-S(A)	
	lock-S(B)
	Read(B)
read(A)	
lock-X(B)	

	lock-X(A)

4、考虑时间戳排序协议，以及两个事务，一个执行写两个数据项 p 和 q，另一个执行读这两个数据项，试给出一个调度，使得第一个事务写操作的时间戳测试失败，引起该事务重启，并依次引起另一个事务的级联中止。并说明是怎样导致这两个事务都饿死的。

解：

T_1	T_2
write(p)	
	read(p)
	read(q)
write(q)	

让 $TS(T_1) < TS(T_2)$ 并让除 write(q) 之外的每个操作的时间戳测试成功。 当事务 T_1 对 write(q) 进行时间戳测试时，它发现 $TS(T_1) <$ R-timestamp(q)，因为 $TS(T_1) < TS(T_2)$ 并且 R-timestamp(q) = $TS(T_2)$。 因此写操作失败，事务 T1 回滚。因为事务 T2 使用由事务 T_1 写入的数据项 p 的值，导致事务 T_2 也被回滚。如果每次重新启动事务时都完全重复这种情况，则可能导致两个事务都处于饥饿状态。

5、设 T1、T2、T3 是如下三个事务：

T1：A:=A+4

T2：A:=A*3

T3：A:=A^2

初始 A=2

设三个事务都遵守两段锁协议，按 T2-T3-T1 的顺序执行，请给出一个不产生死锁的可串行化调度（在下表中填写相应代码即可），并给出最终 A 的结果

时间	T1	T2	T3
1		LOCK-X(A)	
2		READ(A)	
3		A:=A*3	
4		WRITE(A)	
5		UNLOCK(A)	
6			LOCK-X(A)
7			READ(A)
8			A:=A^2
9			WRITE(A)
10			UNLOCK(A)
11	LOCK-X(A)		
12	READ(A)		
13	A:=A+4		
14	WRITE(A)		

15	UNLOCK(A)		

A=40

6、一个带检查点的日志内容如下，结束处发生了故障，请简述恢复算法的过程，并给出 Undo-List 和 Redo-List，以及数据库系统恢复后的 A,B,C 的值。

Start of the logs
…
<T4 start>
<T4, B, 1000, 1700>
<T5 start>
<checkpoint {T4, T5}>
<T5, C, 300, 100>
<T5 commit>
<T6 start>
<T6, A, 700, 600>
<T4, B, 1000>
<T4 abort>
←System crash, start recovery

找到最后的 checkpoint，令 Undo-List=L，本题中为{T4,T5}，从此处向下扫描，若是 start 的事务则加入 Undo-List，若是 commit 或 abort 的事务则从 Undo-List 移除，并添加到 Redo-List。将 Undo-List 中事务涉及的 ABC 赋旧值，将 Redo-List 中的事务重新执行，即对 ABC 赋新值。

Undo-List：T6

Redo-List：T4，T5

A=700，B=1000，C=100

7、有一个带检查点的日志内容如下，请完善日志。

<T0 start> //T0 事务开始
<T0,A,-30,10>
<T0, commit> //T0 事务提交
<T1, start> //T1 事务开始
<T1,B,-10,10>
<T2, start> //T2 事务开始
<T2,A,10,20>
<T2,A,__20,-20>
<checkpoint{__T1,__T2}> //检查点标记
<T2, commit> //T2 事务提交
<T3, start> //T3 事务开始
<T3,C,10,-20>
<T3,D,-20,30>

结束处<T3,D,-20,30>发生了故障，请问恢复后数据库中 A,B,C,D 的值各为多少？ (注：更新日志记录的格式为< Ti, X, Vold, Vnew>)

A=-20，B=-10，C=10，D=-20

8、请解释检查点机制的目的，应该间隔多长时间做一次检查点呢，执行检查点的频率在以下各种情况下分别有何影响。

（1）无故障发生时。

（2）从系统崩溃中恢复。

（3）从介质（磁盘）故障中恢复。
答：
检查点是通过基于日志的恢复方案完成的，以减少崩溃后恢复所需的时间。 如果没有检查点，则必须在崩溃后搜索整个日志，并从日志中撤消/重做所有事务。如果已执行检查点，则在恢复时可以忽略检查点之前的大部分日志记录。执行检查点的另一个原因是在稳定存储已满时清除日志记录。由于检查点在执行过程中会导致一些性能损失，因此如果快速恢复不重要，则应降低其频率。如果我们需要快速恢复，检查点频率应该增加。如果可用的稳定存储量较少，那么频繁的检查点是不可避免的。检查点对从磁盘崩溃中恢复没有影响，存档转储相当于从磁盘崩溃中恢复的检查点。

九、
已知 2 个关系 R(A, B)和 S(B, C)，其主键分别为 R.A 和 S.B。R 有 20000 个元组，S 有 60000 个元组，一块中可以容纳 20 个 R 元组或 30 个 S 元组。设 2 个关系均采用聚簇存储，且每个关系中的元组均已按照其主键值递增排序。现在要执行自然连接操作 R ⋈ S。设缓冲区中可用内存页数为 M = 41。回答下列问题：

1. 采用嵌套循环连接算法执行 R ⋈ S 分别需要进行多少次 I/O？给出具体分析过程。

答:
使用 R 作为外部关系，我们需要 20000 ＊ (60000 + 1) =
1200020000 次 I/O, 如果 S 是外部关系, 我们需要 60000 *(20000+1)=1200060000 次 I/O。
（写一种就行）

2. 采用归并连接算法执行 R ⋈ S 分别需要进行多少次 I/O？给出具体分析过程。

根据题意，外存中存储 R 的结果需要占用 20000/20=1000 个块。
外存中存储 S 的结果需要占用 60000/30=2000 个块。

算法第一阶段 I/O 代价分析：因为 R 中元组未按属性 R.B 进行排序，所以首先需要对 R 建立归并段，需要执行 2B（R）=2000 次 IO。因为 S 中的元组已经按照属性 S.B 进行排序，所以无需对 S 建立归并段。因此算法第一阶段需要执行 B(S)=2000 次 IO。

算法第二阶段的 IO 代价分析：因为 M-1=40，所以 R 的归并段数量为 B（R）/40=25。算法第二阶段可以将 R 的全部归并段与 S 进行归并，需要进行 B(R)+B（S）=3000 次 IO
因此，该算法总共执行 5000 次 IO。

3. 设 R.B 是关系 R 的外键，参照 S.B。如果 R ⋈ S 的结果中元组的平均大小是 R 中元组平均大小的 1.2 倍，R ⋈ S 的结果中元组的平均大小是 S 中元组平均大小的 2 倍，那么在外存中存储 R ⋈ S 的结果需要占用多少个块（页）？给出具体分析过程。

因为 R.B 是关系 R 的外键，所以 R ⋈ S 的结果中包含的元组数一定与 R 的元组数相同。
因为 R ⋈ S 的结果中元组的平均大小是 R 的 1.2 倍，则 1.2*（20000/20）=1200 个块。